

DevFlow

Minimal Development Guideline — Real-Time Team Collaboration Platform

1. Project Goal

Build a Real-Time Team Collaboration App for a single user:

- Login/Register
- Workspace
- Project
- Task manage
- Team member- real-time chat
- Online/offline status
- Real-time notification

2. Tech Stack

Frontend: Next.js, React, TypeScript, Tailwind CSS, Socket.IO Client

Backend: Node.js, Express.js, TypeScript, Socket.IO, JWT

Database: MongoDB + Mongoose

Real-Time: Redis

3. Basic Architecture

Next.js / React → Node.js + Express → Socket.IO → Redis → MongoDB

MongoDB → permanent application data

Redis → presence/cache/pub-sub

Socket.IO → real-time communication

4. Development Steps

Step 1 — Project Setup

Create devflow/frontend/ and devflow/backend/. Setup Git repository and environment variables.

Step 2 — Backend

Setup Express, TypeScript, MongoDB, Mongoose, JWT, Socket.IO and Redis. Create GET /api/health.

Step 3 — Authentication

Implement Register, Login, Logout and Get Current User using JWT.

Step 4 — Workspace & Project

Create User, Workspace, Project and Task models. Implement basic CRUD.

Step 5 — Frontend

Create /login, /register, /dashboard and /projects/[id]. Show projects, tasks, team members and notifications.

Step 6 — Socket.IO

Connect frontend with backend. Implement connection/disconnection and project rooms such as project:{projectId}.

Step 7 — Real-Time Chat

Implement message: send and message: new. Save messages in MongoDB and deliver them through Socket.IO.

Step 8 — Typing & Presence

Add `typing:start`, `typing:stop`, `user:online` and `user:offline`. Use Redis for online users.

Step 9 — Real-Time Task Updates

Update MongoDB, emit a Socket.IO event, and update all project members without page refresh.

Step 10 — Notifications

Implement Task Assigned, Task Updated and New Message notifications. Store them in MongoDB and deliver instantly with Socket.IO.

Step 11 — Redis

Use Redis for online presence, Socket.IO Pub/Sub and basic caching. Add the Socket.IO Redis Adapter when multiple backend servers are required.

Step 12 — Testing

Test authentication, task CRUD, chat, typing indicator, online/offline status, notifications and socket reconnection.

Step 13 — Deployment

Use Docker, Nginx, MongoDB, Redis, Node.js and Next.js.

5. Production Architecture

User → Nginx → Next.js → Node.js + Socket.IO → Redis / MongoDB

6. MVP Features

- ✓ Authentication
- ✓ Workspace
- ✓ Project
- ✓ Task Management
- ✓ Team Members
- ✓ Real-Time Chat
- ✓ Typing Indicator
- ✓ Online/Offline Status
- ✓ Real-Time Notifications
- ✓ Redis
- ✓ MongoDB
- ✓ Socket.IO

7. Development Principle

First build the normal CRUD application → then add Socket.IO → then add Redis → finally deploy.

Keep the first version focused. Do not add file uploads, advanced analytics, email notifications, complex permissions, or other large features until the MVP is stable.